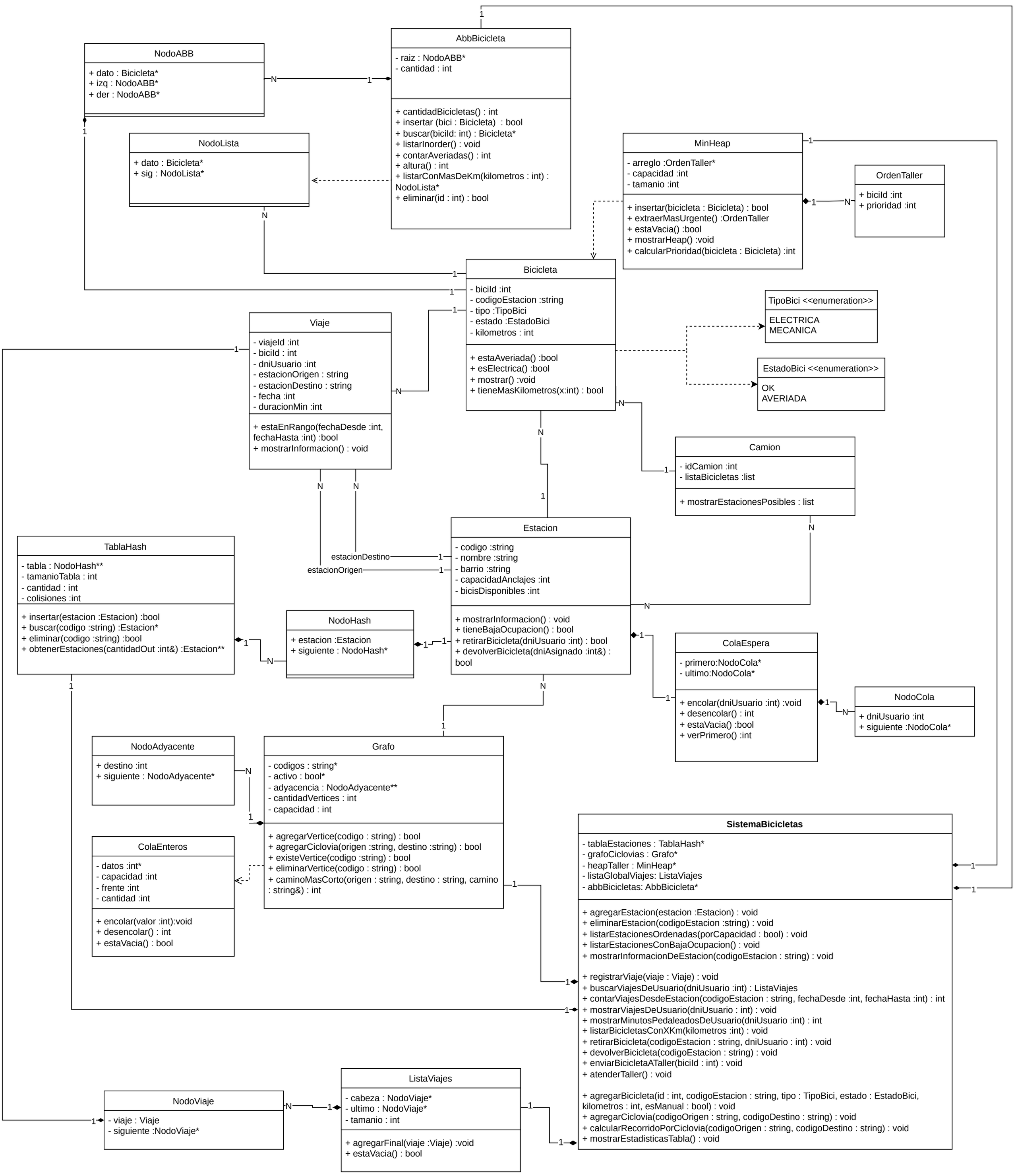

NodoABB
+ dato : Bicicleta*
+ izq : NodoABB*
+ der : NodoABB*
AbbBicicleta
- raiz : NodoABB*
- cantidad : int
+ cantidadBicicletas() : int
+ insertar (bici : Bicicleta) : bool
+ buscar(biciId: int) : Bicicleta*
+ listarInorder() : void
+ contarAveriadas() : int
+ altura() : int
+ listarConMasDeKm(kilometros : int) : NodoLista*
+ eliminar(id : int) : bool
NodoLista
+ dato : Bicicleta*
+ sig : NodoLista*
MinHeap
- arreglo :OrdenTaller*
- capacidad :int
- tamanio :int
+ insertar(bicicleta : Bicicleta) : bool
+ extraerMasUrgente() :OrdenTaller
+ estaVacia() :bool
+ mostrarHeap() :void
+ calcularPrioridad(bicicleta : Bicicleta) :int
OrdenTaller
+ biciId :int
+ prioridad :int
Bicicleta
- biciId :int
- codigoEstacion :string
- tipo :TipoBici
- estado :EstadoBici
- kilometros : int
+ estaAveriada() :bool
+ esElectrica() :bool
+ mostrar() :void
+ tieneMasKilometros(x:int) : bool
TipoBici <<enumeration>>
ELECTRICA
MECANICA
EstadoBici <<enumeration>>
OK
AVERIADA
Viaje
- viajeId :int
- biciId : int
- dniUsuario :int
- estacionOrigen : string
- estacionDestino : string
- fecha :int
- duracionMin :int
+ estaEnRango(fechaDesde :int, fechaHasta :int) :bool
+ mostrarInformacion() : void
Camion
- idCamion :int
- listaBicicletas :list
+ mostrarEstacionesPosibles : list
Estacion
- codigo :string
- nombre :string
- barrio :string
- capacidadAnclajes :int
- bicisDisponibles :int
+ mostrarInformacion() : void
+ tieneBajaOcupacion() : bool
+ retirarBicicleta(dniUsuario :int) : bool
+ devolverBicicleta(dniAsignado :int&) : bool
estacionDestino
estacionOrigen
TablaHash
- tabla : NodoHash**
- tamanioTabla : int
- cantidad : int
- colisiones :int
+ insertar(estacion :Estacion) :bool
+ buscar(codigo :string) :Estacion*
+ eliminar(codigo :string) :bool
+ obtenerEstaciones(cantidadOut :int&) :Estacion**
NodoHash
+ estacion :Estacion
+ siguiente : NodoHash*
ColaEspera
- primero:NodoCola*
- ultimo:NodoCola*
+ encolar(dniUsuario :int) :void
+ desencolar() : int
+ estaVacia() :bool
+ verPrimero() :int
NodoCola
+ dniUsuario :int
+ siguiente :NodoCola*
NodoAdyacente
+ destino :int
+ siguiente : NodoAdyacente*
Grafo
- codigos : string*
- activo : bool*
- adyacencia : NodoAdyacente**
- cantidadVertices : int
- capacidad : int
+ agregarVertice(codigo : string) : bool
+ agregarCiclovia(origen :string, destino :string) : bool
+ existeVertice(codigo :string) : bool
+ eliminarVertice(codigo : string) : bool
+ caminoMasCorto(origen : string, destino : string, camino : string&) : int
ColaEnteros
- datos :int*
- capacidad :int
- frente :int
- cantidad :int
+ encolar(valor :int):void
+ desencolar() : int
+ estaVacia() : bool
SistemaBicicletas
- tablaEstaciones : TablaHash*
- grafoCiclovias : Grafo*
- heapTaller : MinHeap*
- listaGlobalViajes: ListaViajes
- abbBicicletas: AbbBicicleta*
+ agregarEstacion(estacion :Estacion) : void
+ eliminarEstacion(codigoEstacion :string) : void
+ listarEstacionesOrdenadas(porCapacidad : bool) : void
+ listarEstacionesConBajaOcupacion() : void
+ mostrarInformacionDeEstacion(codigoEstacion : string) : void
+ registrarViaje(viaje : Viaje) : void
+ buscarViajesDeUsuario(dniUsuario :int) : ListaViajes
+ contarViajesDesdeEstacion(codigoEstacion : string, fechaDesde :int, fechaHasta :int) : int
+ mostrarViajesDeUsuario(dniUsuario : int) : void
+ mostrarMinutosPedaleadosDeUsuario(dniUsuario :int) : int
+ listarBicicletasConXKm(kilometros :int) : void
+ retirarBicicleta(codigoEstacion : string, dniUsuario : int) : void
+ devolverBicicleta(codigoEstacion : string) : void
+ enviarBicicletaATaller(biciId : int) : void
+ atenderTaller() : void
+ agregarBicicleta(id : int, codigoEstacion : string, tipo : TipoBici, estado : EstadoBici, kilometros : int, esManual : bool) : void
+ agregarCiclovia(codigoOrigen : string, codigoDestino : string) : void
+ calcularRecorridoPorCiclovia(codigoOrigen : string, codigoDestino : string) : void
+ mostrarEstadisticasTabla() : void
NodoViaje
- viaje : Viaje
- siguiente :NodoViaje*
ListaViajes
- cabeza : NodoViaje*
- ultimo : NodoViaje*
- tamanio : int
+ agregarFinal(viaje :Viaje) :void
+ estaVacia() : bool